

Introducción a la Calidad y Pruebas de Software

Curso: Introducción a la Programación II · Ingeniería en Sistemas



Agenda de la Clase

Exploraremos los fundamentos de la calidad del software, desde conceptos básicos hasta técnicas modernas de pruebas en entornos ágiles.

01

Calidad de Software

Definición, importancia y relación con el ciclo de vida

02

Conceptos de Pruebas

Error, defecto, fallo y principios básicos

03

Casos de Prueba

Estructura y ejemplos prácticos en C#

04

Validación y Manejo de Errores

Técnicas con condicionales, Try/Catch y excepciones

05

Pruebas en Entornos Ágiles

Unitarias, integración, seguridad y Shift Left Testing

¿Qué es la Calidad de Software?

Definición

La calidad del software es el grado en que un sistema satisface los requisitos especificados, las necesidades del usuario y las expectativas implícitas del cliente. Abarca atributos como funcionalidad, confiabilidad, eficiencia, usabilidad y mantenibilidad.

¿Por qué importa?

- Un software de baja calidad genera pérdidas económicas y reputacionales
- Las pruebas detectan fallos **antes** de llegar al usuario final
- El costo de corregir un error en producción es hasta **100x mayor** que en etapas tempranas
- La satisfacción del usuario depende directamente de la estabilidad del sistema

📄 Ejemplo clásico: el fallo del cohete Ariane 5 en 1996 se debió a un error de software no detectado, costando más de 370 millones de dólares.

Conceptos Fundamentales de Pruebas

Error

Equivocación cometida por el programador al escribir el código. Es la **causa raíz** del problema.

Ejemplo: usar `+` en lugar de `-` en un cálculo.

Defecto (Bug)

Manifestación del error en el código fuente o en los artefactos del sistema. Es la **imperfección detectada** durante revisión o prueba.

Fallo

Comportamiento incorrecto observable cuando el sistema se ejecuta. Es la **consecuencia visible** del defecto en tiempo de ejecución.

Ejemplo en C#: un posible error y su prueba

```
// Método con error lógico
public static int Dividir(int a, int b) {
    return a + b; // ERROR: debería ser a / b
}

// Prueba que detecta el error
[Fact]
public void Dividir_DebeRetornarCociente() {
    int resultado = Dividir(10, 2);
    Assert.Equal(5, resultado); // FALLA: retorna 12 en vez de 5
}
```

Objetivos de las Pruebas de Software

Las pruebas no solo buscan encontrar errores; tienen múltiples objetivos estratégicos que impactan directamente en la calidad y el costo del producto final.

Objetivo	Beneficio	Ejemplo Práctico
Encontrar errores antes de producción	Reducción drástica del costo de corrección	Detectar división por cero en etapa de desarrollo
Garantizar funcionamiento esperado	Cumplimiento de requisitos del cliente	Verificar que el login acepta solo credenciales válidas
Mejorar la confiabilidad	Software estable bajo condiciones diversas	Pruebas de estrés en un sistema de pagos
Reducir costos de mantenimiento	Menos incidencias en producción y parches urgentes	Suite de pruebas regresión automática en CI/CD

Casos de Prueba

Elementos de un Caso de Prueba

Identificador

Código único: CP-001

Descripción

Qué se está probando

Datos de Entrada

Valores usados en la prueba

Resultado Esperado

Lo que el sistema debería producir

Resultado Obtenido

Lo que el sistema realmente produjo

Estado

✅ Aprobado / ❌ Fallido

Ejemplo: Calculadora en C#

ID	CP-001
Descripción	Verificar suma de dos enteros positivos
Entrada	a = 5, b = 3
Esperado	8
Obtenido	8
Estado	✅ Aprobado

```
// Método en C#  
public int Sumar(int a, int b) => a + b;
```

```
// Prueba  
Assert.Equal(8, Sumar(5, 3));
```

Validación de Entradas y Manejo de Errores

Validar los datos que el usuario ingresa es una de las primeras líneas de defensa contra errores, fallos de seguridad y comportamientos inesperados. En C# disponemos de varias técnicas.



Condicionales

```
if (edad < 0 || edad > 120)
    throw new ArgumentException(
        "Edad inválida");
```



Try / Catch

```
try {
    int n = int.Parse(input);
} catch (FormatException ex) {
    Console.WriteLine(
        "Dato no numérico: " +
        ex.Message);
}
```



Excepción Personalizada

```
public class EdadInvalidaException
    : Exception {
    public EdadInvalidaException()
        : base("Edad fuera de rango.")
    {}
}
// Uso:
throw new EdadInvalidaException();
```

i Siempre valida: datos obligatorios (nulos/vacíos), rangos permitidos (0–120 para edad), formatos (correo, teléfono) y tipos de datos correctos antes de procesar.



SECCIÓN 1.5

Pruebas en Entornos Ágiles

Pruebas Tradicionales

- Se ejecutan al **final** del ciclo de desarrollo
- Equipo de QA separado del equipo de desarrollo
- Documentación extensa previa a las pruebas
- Corrección de errores costosa y tardía

Agile Testing

- Las pruebas ocurren **durante todo el sprint**
- Testers, devs y PO colaboran activamente
- Pruebas automatizadas integradas en el pipeline CI/CD
- Feedback rápido y corrección inmediata de defectos

En Agile, el tester no espera al final para probar: diseña casos de prueba desde los criterios de aceptación de las historias de usuario y los ejecuta en paralelo con el desarrollo.

Pruebas Unitarias

¿Qué son?

Las pruebas unitarias validan el comportamiento de la **unidad más pequeña** del código: un método o función de forma aislada, sin dependencias externas.

Ventajas

- Detectan errores de forma temprana y precisa
- Facilitan la refactorización segura del código
- Sirven como documentación ejecutable
- Se automatizan fácilmente con xUnit o NUnit

Ejemplo con xUnit en C#

```
// Clase a probar
public class Calculadora {
    public double Dividir(double a, double b) {
        if (b == 0)
            throw new DivideByZeroException();
        return a / b;
    }
}

// Clase de prueba con xUnit
public class CalculadoraTests {
    [Fact]
    public void Dividir_ValoresValidos_RetornaCociente() {
        var calc = new Calculadora();
        double resultado = calc.Dividir(10, 2);
        Assert.Equal(5, resultado);
    }

    [Fact]
    public void Dividir_PorCero_LanzaExcepcion() {
        var calc = new Calculadora();
        Assert.Throws<DivideByZeroException>(
            () => calc.Dividir(10, 0));
    }
}
```

Pruebas de Integración

Las pruebas de integración verifican que **múltiples componentes funcionan correctamente en conjunto**. A diferencia de las pruebas unitarias que aíslan cada pieza, aquí se prueban las interacciones reales entre clases, servicios o bases de datos.

Integración entre Clases

Verificar que `PedidoService` calcula correctamente el total usando `ProductoRepository`.

Integración con BD

Confirmar que al guardar un registro con `Entity Framework`, éste persiste correctamente en la base de datos.

Integración de Servicios

Probar que la API REST devuelve el JSON correcto al consumir un endpoint con datos reales.

```
// Prueba de integración: Servicio + Repositorio
[Fact]
public void ObtenerTotal_PedidoConProductos_RetornaMontoCorrect() {
    var repo = new ProductoRepository(dbContextReal);
    var service = new PedidoService(repo);
    decimal total = service.CalcularTotal(pedidoId: 1);
    Assert.True(total > 0);
}
```

Pruebas de Seguridad Básica

⚠ Validación Insuficiente

No verificar que los datos de entrada sean seguros permite ataques de inyección SQL o XSS. Toda entrada del usuario debe sanitizarse.

🔑 Contraseñas Expuestas

Almacenar contraseñas en texto plano o usar algoritmos débiles (MD5) compromete a todos los usuarios. Siempre usar bcrypt o PBKDF2.

📋 Información Expuesta

Mostrar mensajes de error detallados (stack traces) al usuario revela la arquitectura interna del sistema. Usar logs internos, mensajes genéricos al exterior.

⚠ Un sistema funcional pero inseguro puede causar pérdidas de datos, violaciones de privacidad y daños legales graves. La seguridad no es opcional.



Análisis Estático de Código (SAST)

¿Qué es SAST?

Static Application Security Testing analiza el **código fuente sin ejecutarlo**. Actúa como un "corrector ortográfico de seguridad" que examina el código en busca de patrones vulnerables antes del despliegue.

Beneficios clave

- Detección temprana de vulnerabilidades
- Integrable en el IDE o pipeline CI/CD
- Cobertura de todo el código base
- Sin necesidad de ejecutar la aplicación

Problemas que detecta SAST

```
//
```

Pruebas Dinámicas (DAST)

SAST — Estático

Analiza el **código fuente** sin ejecutar la aplicación. Es como revisar el plano de un edificio buscando fallas estructurales antes de construirlo.

- Se aplica durante el desarrollo
- Requiere acceso al código fuente
- Detecta vulnerabilidades en lógica interna

DAST — Dinámico

Prueba la **aplicación en ejecución** simulando ataques externos reales. Es como intentar entrar al edificio ya construido buscando puertas sin candado.

- Se aplica sobre la aplicación desplegada
- No requiere acceso al código fuente
- Detecta fallos en tiempo de ejecución y APIs

Ejemplos de pruebas DAST: enviar entradas maliciosas a formularios web (*SQL injection*, *XSS*), verificar respuestas HTTP con cabeceras de seguridad y probar autenticación con credenciales inválidas. Herramientas populares: **OWASP ZAP**, **Burp Suite**.

Agile Testing: Principios y Práctica



Colaboración Continua

Desarrolladores, testers y usuarios trabajan juntos desde el inicio del sprint, definiendo criterios de aceptación claros para cada historia de usuario.



Automatización

Las pruebas repetitivas se automatizan con frameworks como xUnit, Selenium o RestSharp, liberando tiempo para pruebas exploratorias de mayor valor.



Entrega Continua

Cada commit dispara una suite de pruebas automáticas en el pipeline CI/CD. Solo el código que pasa todas las pruebas avanza al entorno de producción.



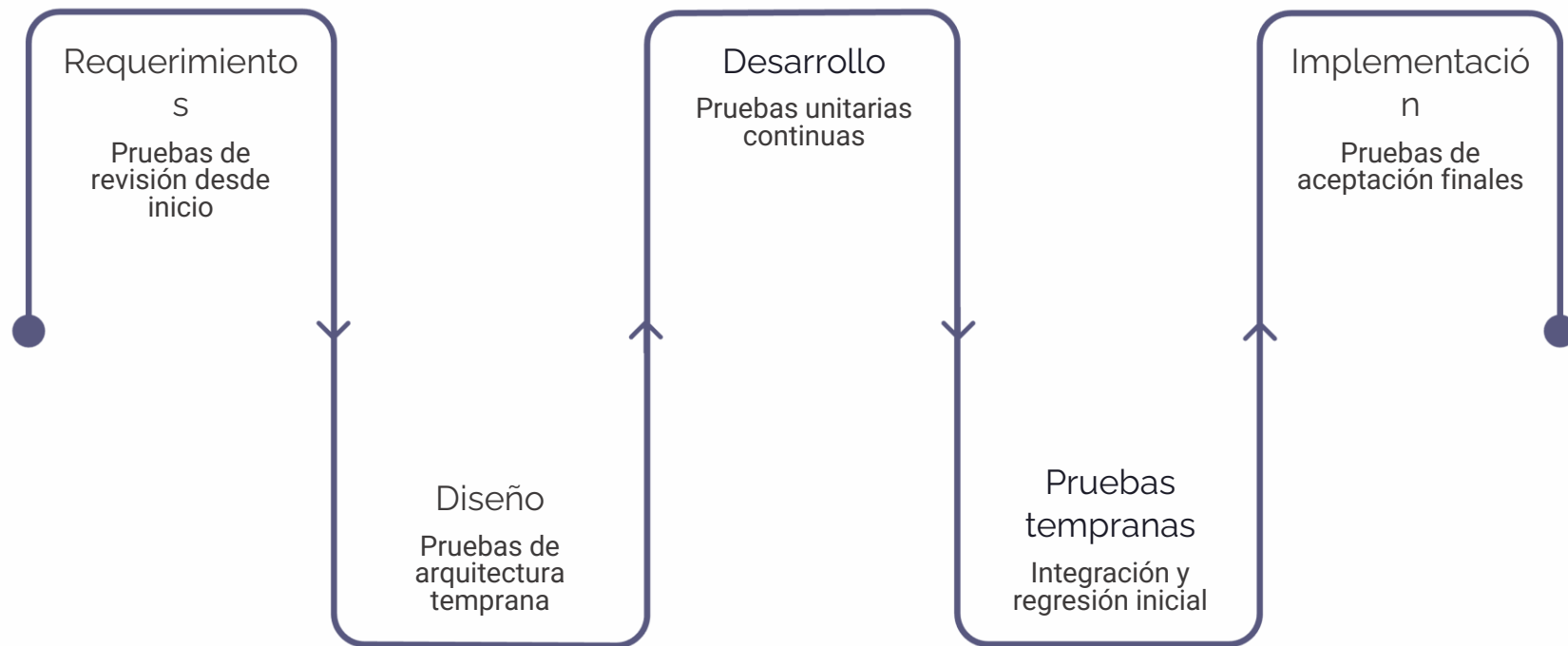
Feedback Rápido

Los resultados de las pruebas están disponibles en minutos. El equipo corrige defectos mientras el contexto del código aún está fresco en la memoria.



Shift Left Testing

Shift Left significa mover las actividades de prueba hacia la **izquierda del ciclo de desarrollo**, es decir, iniciarlas lo más temprano posible, desde los requisitos mismos.



🔥 Menor Costo

Corregir un error en requisitos cuesta 5x menos que hacerlo en producción.

🏆 Mejor Calidad

Los defectos se detectan cuando son más fáciles de comprender y corregir.

⚡ Desarrollo Rápido

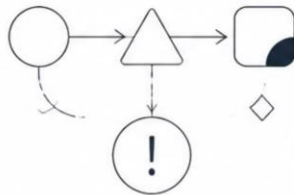
Menos bugs tardíos significa menos interrupciones y entregas más predecibles.

Conceptos Clave de la Clase

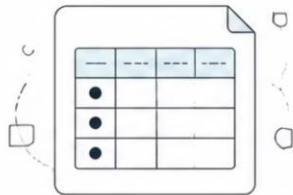
- 1 Calidad de Software:**
grado de satisfacción
de requisitos y
expectativas



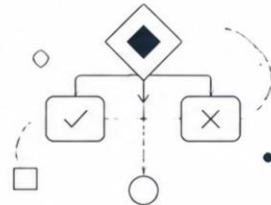
- 2 Error/Defecto/Fallo:**
causa, manifestación
y consecuencia



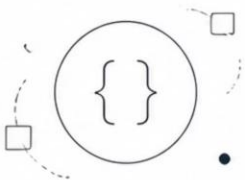
- 3 Casos de Prueba:**
documentos con ID,
entrada, resultado
esperado y estado



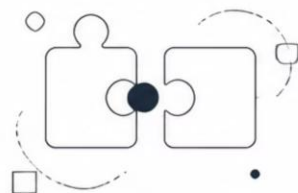
- 4 Validación de Entradas:**
condicionales,
try-catch, excepciones
personalizadas



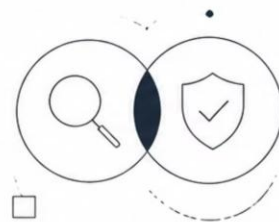
- 5 Pruebas Unitarias:**
validan métodos
individuales con
xUnit/NUnit



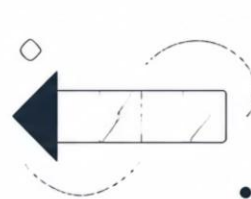
- 6 Pruebas de Integración:**
validan la interacción
entre componentes



- 7 SAST y DAST:**
análisis estático y
dinámico de seguridad



- 8 Shift Left Testing:**
pruebas tempranas
para reducir costos



Preguntas de Discusión

Reflexiona sobre las siguientes preguntas antes de la próxima sesión. Puedes publicar tus respuestas en el foro del curso.

1

Error vs. Fallo

¿Es posible que exista un defecto en el código y que el sistema nunca falle? ¿En qué condiciones ocurriría esto?

2

Costo de las Pruebas

Si las pruebas cuestan tiempo y dinero, ¿por qué es más económico probar que no probar? Da un ejemplo de tu propia experiencia.

3

Seguridad en el Código

¿Qué consecuencias tendría para un banco si sus aplicaciones no utilizaran SAST ni DAST? ¿Quién sería responsable?

4

Shift Left en la Práctica

¿Cómo cambiaría tu forma de programar si tuvieras que escribir primero la prueba y luego el código (TDD – Test Driven Development)?

Ejercicio en C#: Sistema de Registro de Estudiantes

Implementa la clase `Estudiante` con validaciones y escribe al menos **tres casos de prueba con xUnit** que cubran los escenarios indicados.

Instrucciones

1. Crea una clase `Estudiante` con propiedades: `Nombre`, `Edad`, `Carné` y `Promedio`
2. Valida que la `Edad` esté entre 16 y 60 años
3. Valida que el `Promedio` sea entre 0.0 y 10.0
4. El `Nombre` no puede ser nulo o vacío
5. Lanza excepciones personalizadas para cada validación
6. Escribe las pruebas unitarias con xUnit

Escenarios a probar

- ☒ Registro exitoso con datos válidos
- ☒ Edad fuera del rango permitido
- ☒ Promedio negativo o mayor a 10
- ☒ Nombre nulo o en blanco

Estructura de código sugerida

```
public class Estudiante {  
    public string Nombre { get; set; }  
    public int Edad { get; set; }  
    public double Promedio { get; set; }  
  
    public Estudiante(string nombre,  
        int edad,  
        double promedio) {  
        if (string.IsNullOrEmpty(nombre))  
            throw new ArgumentException(  
                "Nombre no puede estar vacío.");  
        if (edad < 16 || edad > 60)  
            throw new ArgumentOutOfRangeException(  
                "Edad debe estar entre 16 y 60.");  
        if (promedio < 0 || promedio > 10)  
            throw new ArgumentOutOfRangeException(  
                "Promedio debe estar entre 0 y 10.");  
        Nombre = nombre;  
        Edad = edad;  
        Promedio = promedio;  
    }  
}  
  
// Tu turno: escribe las pruebas con xUnit  
public class EstudianteTests {  
    [Fact]  
    public void Constructor_DatosValidos_CreaEstudiante(){  
        // Arrange, Act, Assert – ¡completalo!  
    }  
}
```



CIERRE DE CLASE

Entregable y Criterios de Evaluación

1

Código fuente

Subir el proyecto C# completo con la clase `Estudiante` y sus validaciones al repositorio del curso.

2

Casos de prueba documentados

Entregar una tabla con al menos 4 casos de prueba formales (ID, descripción, entrada, esperado, obtenido, estado).

3

Pruebas unitarias con xUnit

Incluir el proyecto de pruebas con al menos 4 métodos `[Fact]` ejecutables y con resultado **verde**.

4

Reflexión escrita

Responder en el foro al menos una de las preguntas de discusión con un párrafo argumentado (mínimo 80 palabras).



Fecha de entrega: una semana a partir de hoy. Consultas al foro del módulo o por mensaje privado en la plataforma del curso. ¡Mucho éxito!

Lecturas y Herramientas Recomendadas

Bibliografía

- Sommerville, I. — *Ingeniería del Software*, 10ª ed. Capítulo 8: Pruebas de software
- Myers, G. — *The Art of Software Testing*, 3ª ed.
- Beck, K. — *Test-Driven Development: By Example*
- ISTQB Foundation Level — Glosario y Syllabus oficial (disponible en [istqb.org](https://www.istqb.org))

Herramientas para practicar

xUnit / NUnit

Frameworks de pruebas unitarias para C# y .NET

SonarQube / Roslyn

Análisis estático de calidad y seguridad del código

OWASP ZAP

Herramienta gratuita de pruebas dinámicas de seguridad

GitHub Actions

Pipeline CI/CD gratuito para automatizar pruebas en cada commit